Review notes · for you

Let the computer search: unit check

Optimization · Level 3 · Unit check

Time	Materials
20 min	Paper and a pen. No device needed.

Learning goals

- Try them all, or think ahead
- Shortest paths
- The bus route
- Timetables with CP-SAT

Activity: Search race

1. Warm-up (3 min): For each lesson, write one search from your life that it could solve: packing, a way to school, a round trip, a timetable.
2. Race (10 min): With a partner, pick one small problem. One of you solves it with a quick rule (greedy or nearest neighbour), the other with a sure method (every option, Dijkstra, or a small model).
3. Compare (4 min): Compare your answers and how long each took. When did the quick rule miss?
4. Fair (3 min): Look at the best answer and ask who it would be hard on: the first stop, the last slot, the school that got nothing.

Keep the problems small (four or five items, places or tasks) so both methods finish in the time.

Discussion

- When is a quick rule good enough?
- Which method surprised you most?
- Who should decide what “fair” means in a search?

Check yourself

- The method I would trust for my own search + one fairness question I would ask.

Try it this week

Pick one search from your own life (packing a bag, a route you take, a week to plan). Solve it with code or on paper using a method from this unit, then check whether the answer is fair to everyone it affects.

Quiz sheet

Let the computer search: unit check

Optimization · Level 3 · Unit check

Name _____ Class _____ Date _____

1 **A brute-force search over 10 items checks 1,024 possible loads.**

◯ True ◯ False

2 **What does this program print?**

```
w = [3, 2, 2]
v = [5, 3, 3]
best = 0
for mask in range(2 ** 3):
    kg = sum(w[i] for i in range(3) if mask >> i & 1)
    pts = sum(v[i] for i in range(3) if mask >> i & 1)
    if kg <= 4 and pts > best:
        best = pts
print(best)
```

◯ 5 ◯ 6 ◯ 11

3 **What does Dijkstra's method need to give the right answer? Choose all that apply.**

☐ Every road the same length ☐ Places listed in alphabetical order

☐ Settling the smallest waiting number first ☐ No road with a negative length

4 **On the map from the lesson "Shortest paths", this Dijkstra program says the school is 106 km away, but the shortest way is 6. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  while todo:
2      p = min(todo, key=lambda q: dist[q])
3      todo.remove(p)
4      for q, km in roads[p].items():
5          dist[q] = max(dist[q], dist[p] + km)
```

5 **Put in order: solving a timetable with CP-SAT.**

☐ Read the status

☐ Check the result with the people involved

☐ Give each choice a variable with its domain

☐ Run the solver

☐ Write each rule as a line

6 **Complete the sentence.**

A greedy rule is _____ but can _____ the best; dynamic programming is sure because it _____ smaller answers.

Word bank: doubles · reuses · fast · guesses · miss

7 **Complete the club model: a day from 1 to 3 for each club, each student's clubs on different days, and football not on day 3.**

```
day = {c: m._____(1, 3, c) for c in clubs}
for p in pupils:
    m._____([day[c] for c in p])
m.add(day["football"] _____ 2)
```

Word bank: `new_bool_var` `>=` `new_int_var` `add_all_different` `<=` `maximize`

8 **Your nearest-neighbour program prints a 19 km route, and an exact search finds 16 km. What does that tell you?**

◯ A quick rule can miss the best route ◯ The 16 km route must skip a village

◯ Your program added the distances wrongly

9 **Put the lines in order: add up the weight and points of every load in a brute-force search.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

☐ `for i in range(6):`

☐ `if mask >> i & 1:`

☐ `kg, pts = kg + w[i], pts + v[i]`

☐ `for mask in range(2 ** 6):`

☐ `if mask >> i:`

☐ `kg = pts = 0`

10 **If a timetable model with no goal says OPTIMAL, the timetable is the fairest one possible.**

◯ True ◯ False

Check your answers

Let the computer search: unit check

Optimization · Level 3 · Unit check

1 **True**

Each item is in or out, so 10 items give $2^{10} = 1{,}024$ loads.

2 **6**

The two lighter items fit together (4 kg, 6 points); the heavy one alone gives only 5, and all three (11 points) are too heavy.

3 **No road with a negative length; Settling the smallest waiting number first**

Settling the smallest number is safe because every other way is at least as long, which needs road lengths that are never negative.

4 **Line 5: `dist[q] = min(dist[q], dist[p] + km)`**

Relaxing a road must keep the smaller of the old number and the new way: min, not max.

5 **Top to bottom: 4, 5, 1, 3, 2**

First describe the problem (variables, then rules), then let the solver search, then read the status (OPTIMAL, FEASIBLE or INFEASIBLE) before you look at the timetable, and finally judge it with the people it affects.

6 **A greedy rule is fast but can miss the best; dynamic programming is sure because it reuses smaller answers.**

Greedy takes what looks best now and never compares, so it is quick but can miss (60 against 65 on the truck).

7

```
day = {c: m.new_int_var(1, 3, c) for c in clubs}
for p in pupils:
    m.add_all_different([day[c] for c in p])
m.add(day["football"] <= 2)
```

A day is a whole number from 1 to 3 (new_int_var, not a yes/no variable); one all-different rule per student keeps their clubs apart; and football's day must be at most 2, with the expression on the left.

8 **A quick rule can miss the best route**

Nearest neighbour never looks ahead, so its last legs can be long; the 16 km route visits every village too.

9 **Top to bottom: 3, 4, 5, 1, X, 2**

```
for mask in range(2 ** 6):
    kg = pts = 0
    for i in range(6):
        if mask >> i & 1:
            kg, pts = kg + w[i], pts + v[i]
```

Every mask from 0 to 63 is one load; for each, start from zero and add the items whose bit is 1.

10 **False**

With no goal, OPTIMAL only means every rule you wrote is kept.